

Embedded Linux Application with Buildroot and Colour Sensor

From Buildroot generation to remote deployment, I2C enablement, sensor integration and target-side validation on Raspberry Pi.

Buildroot

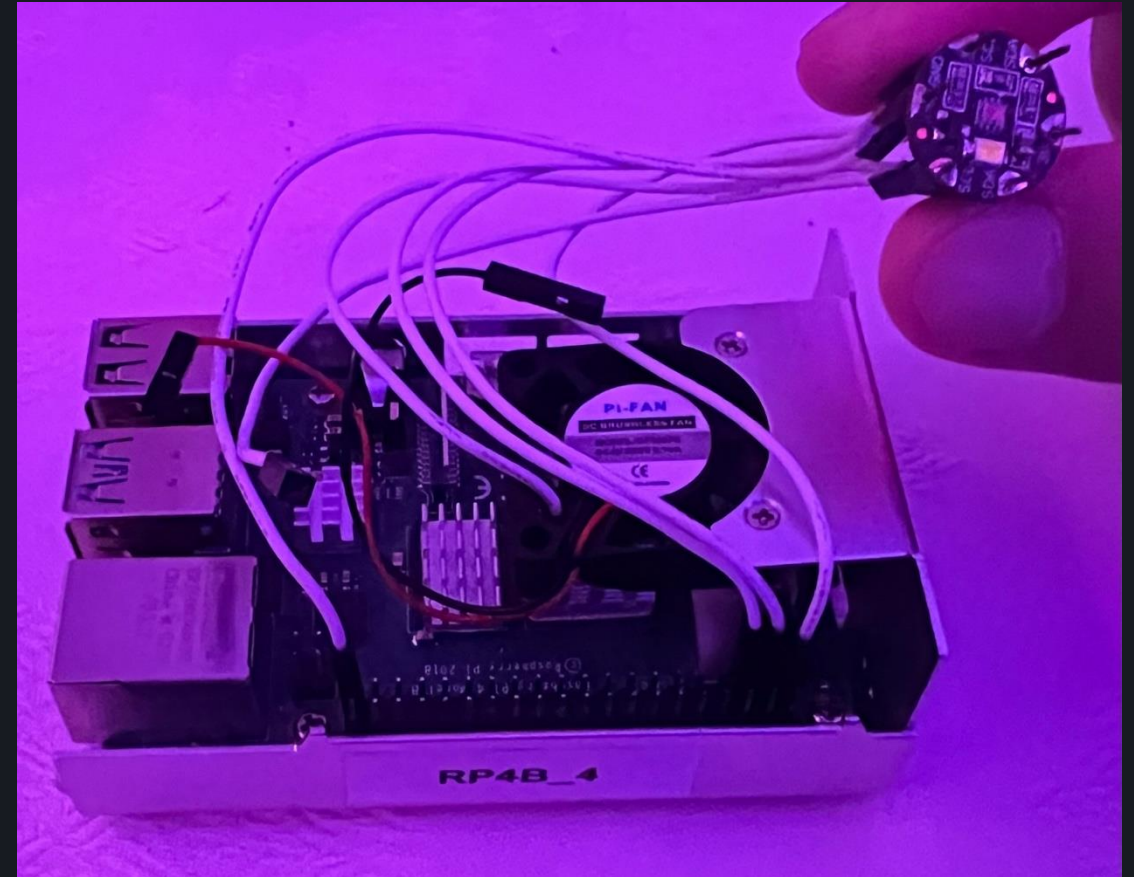
Eclipse

I2C

Color Sensor

Remote test

Team: Covaliov Mocenco Alexandru · Ungureanu Radu Ionuț · Tița Andrei Dragoș
Universidad Politécnica de Madrid · Embedded System Design with Raspberry Pi



Final hardware used for the I2C colour sensing demo

Project overview and presentation map

02

2

Project stages completed

3–4

Chapter scope covered

10–12

Minute presentation

0x29

Detected sensor address

01**Problem and goals**

What Project II asked us to deliver

02**First part**

Buildroot generation and board boot

03**Second part**

Eclipse, cross-compiling and remote run

04**Sensor integration**

I2C enablement, wiring and detection

05**Validation**

Compile, deploy, run, demo and conclusions

What Project II required us to demonstrate

03

Core deliverables

- Generate an embedded Linux system with Buildroot
- Complete the Eclipse remote development workflow
- Implement a colour sensor application on Raspberry Pi
- Run the final executable on the Buildroot target

Evidence used in the presentation

- Assignment screenshot and task breakdown
- Boot and network validation on the Buildroot system
- I2C bus detection with the sensor visible at address 0x29
- Successful compilation, transfer and execution of the final app

First Part — Buildroot system generation

04

01 Configure Buildroot

Target platform, system settings and Linux integration were configured for Raspberry Pi.

02 Compile custom image

We completed the full Buildroot build and generated the embedded Linux image.

03 Deploy and boot

The generated image was written to the SD card and validated on the board.

04 Confirm target access

After boot, we validated shell access, basic runtime behaviour and system readiness.

05 Enable missing interfaces

I2C support required an extra Linux configuration pass and rebuild.

06 Ready for application layer

Once boot + bus access worked, we moved to remote development and sensor software.

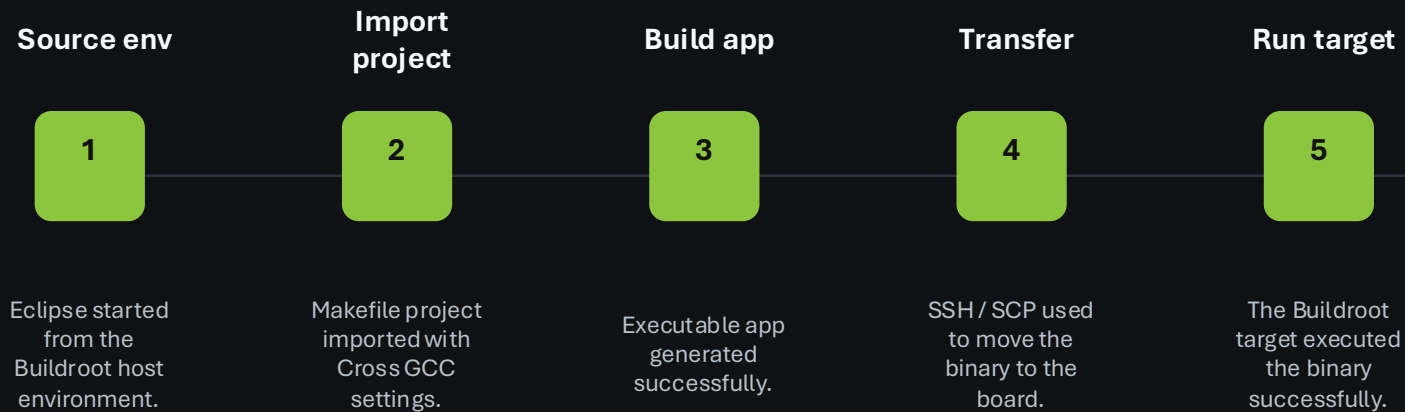
Key commands

```
1 make menuconfig
2 make linux-menuconfig
3 make
4 sync
```

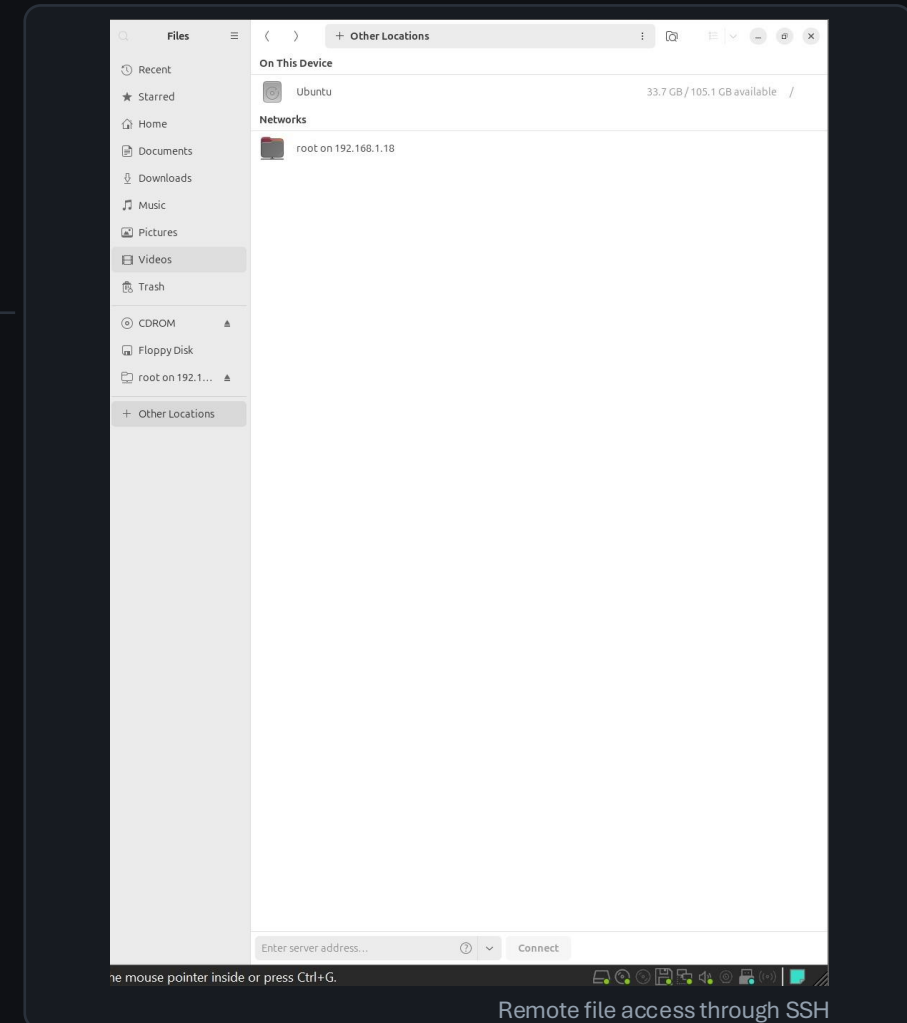
Main outcome: the Raspberry Pi booted the Buildroot-generated Linux system and became a valid target for deployment and testing.

Second Part — Eclipse remote workflow

05

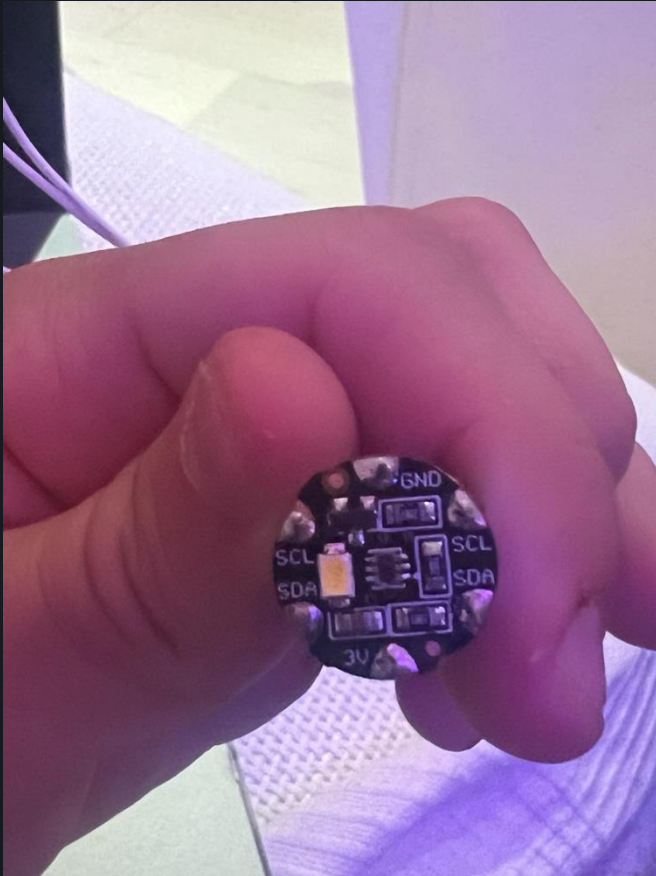


```
1 cd ./build/host
2 source environment-setup
3 /home/ubuntu/eclipse/cpp-2024-09/eclipse/eclipse &
```



Sensor integration and hardware bring-up

06



Colour sensor module

Wiring

- 3V → 3.3V
- GND → GND
- SDA → GPIO2 / SDA
- SCL → GPIO3 / SCL

Bus validation

- 1 `ls /dev/i2c*`
- 2 `i2cdetect -y 1`

Once `/dev/i2c-1` existed and the sensor appeared at `0x29`, software integration could start.

```
Linux RPI 4
buildroot login: [ 8.740343] bcmgenet fd580000.ethernet eth0: Link is Up - 1G
bps/Full - flow control off

Linux RPI 4
buildroot login: root
Password:
# ls /dev/i2c*
/dev/i2c-1
# i2cdetect -y 1
-sh: i2cdetect: not found
# i2cdetect -y 1
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- 29 -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
#
```

I2C success: sensor visible at 0x29

Software implementation — reuse from Project I

We kept the original I2C workflow and only replaced the sensor-specific parts: address, registers, initialization and RGBC reads.

Sensor mapping

```
1 #define TCS34725_ADDR 0x29
2 #define TCS34725_ID   0x12
3 #define TCS34725_RDATAL 0x16
```

I2C bus access

```
1 if ((file_i2c =
open("/dev/i2c-1", O_RDWR)) <
0) return 1;
2 ioctl(file_i2c, I2C_SLAVE,
TCS34725_ADDR);
```

Main acquisition loop

```
1 clear =
read_word(TCS34725_CDATAL);
2 red   =
read_word(TCS34725_RDATAL);
3 green =
read_word(TCS34725_GDATAL);
4 blue  =
read_word(TCS34725_BDATAL);
```

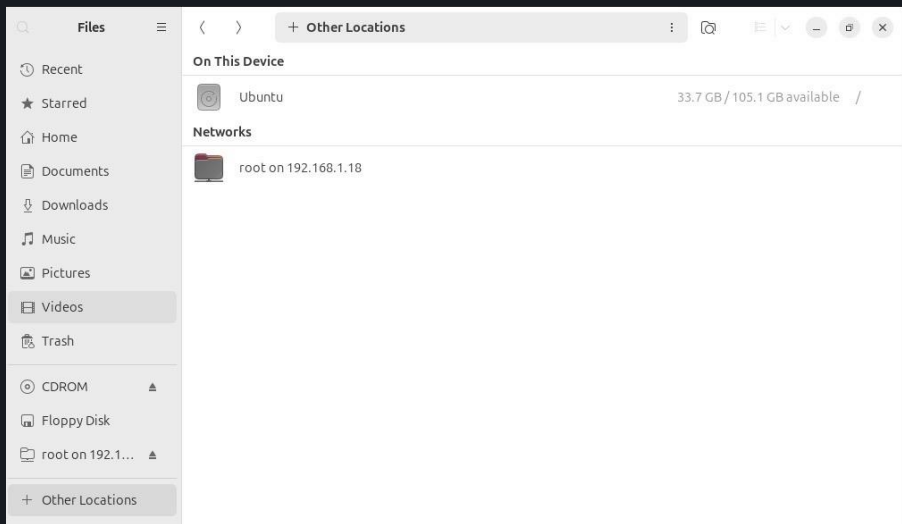
- Reused from Project I: bus opening, ioctl slave selection, register reads, periodic loop, CTRL+C stop handler.
- Added for Project II: Sensor register map, sensor ID check, integration time setup and normalized RGB output.

Compilation, deployment and execution

08

```
[Compile] color_sensor.c
cc -c -g -O0 color_sensor.c -o color_sensor.o
[link] app
cc -o app color_sensor.o -lpthread -lm
```

Successful build output



SSH-based transfer path

Execution chain

```
1 make clean
2 make
3 scp app
root@192.168.1.18:/root/
4 ssh root@192.168.1.18
5 cd /root
6 ./app 1
```

This stage proved that the final executable compiled on the development side and ran on the Buildroot target side.

```
Linux RPI 4
buildroot login: root
Password:
# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq qlen 1000
    link/ether d8:3a:dd:fa:33:34 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.18/24 brd 192.168.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::da3a:ddff:fe3a:3334/64 scope link
        valid_lft forever preferred_lft forever
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether d8:3a:dd:fa:33:35 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.19/24 brd 192.168.1.255 scope global wlan0
        valid_lft forever preferred_lft forever
    inet6 fe80::da3a:ddff:fe3a:3335/64 scope link
        valid_lft forever preferred_lft forever
# ls /root
#
```

Target IP discovery

What we presented by now

- I2C bus enabled and sensor detected
- Final app compiled without errors
- Binary transferred to Raspberry Pi
- Application executed on the Buildroot system

Main difficulties and how we solved them

09

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:0c:29:d4:0e:d8 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.208.128/24 brd 192.168.208.255 scope global dynamic noprefixroute ens33
        valid_lft 1121sec preferred_lft 1121sec
    inet6 fe80::20c:29ff:fed4:ed8/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-VMware-Virtual-Platform:~$
```

VM and target were on different subnets

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/eclipse-workspace/project_eclipse$ scp ~
~/eclipse-workspace/project_eclipse/app root@192.168.1.18:/root/
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@   WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!   @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:lQubAkhEcwFKTzNP//kdaw1RfLtLHdxxebglEzHVsE.
Please contact your system administrator.
Add correct host key in /home/ubuntu/.ssh/known_hosts to get rid of this message
'
Offending ED25519 key in /home/ubuntu/.ssh/known_hosts:4
  remove with:
  ssh-keygen -f '/home/ubuntu/.ssh/known_hosts' -R '192.168.1.18'
Host key for 192.168.1.18 has changed and you have requested strict checking.
Host key verification failed.
scp: Connection closed
ubuntu@ubuntu-VMware-Virtual-Platform:~/eclipse-workspace/project_eclipse$
```

SSH host key mismatch during transfer

01 Network mismatch

The VM initially had a 192.168.208.x address while the board was on 192.168.1.x. We moved both into the same reachable setup before retrying SSH.

02 SSH access issue

Remote copy failed because the saved host key no longer matched the current target. We removed the old key and reconnected cleanly.

03 Missing /dev/i2c-1

The first Buildroot boot did not expose the I2C device. We enabled I2C in Linux config, rebuilt and redeployed the image.

04 Sensor integration risk

We validated the hardware step by step: wiring, bus availability, 0x29 detection, only then final app execution.

Key lesson: validating the infrastructure first (Buildroot boot, SSH, I2C bus, sensor detection) made the final software debugging much faster and safer.

Final results and technical outcome

10

Done

Chapter 3 completed

Done

Chapter 4 completed

0x29

Sensor detected

1 app

Final executable

Live

Ready to demo

What we achieved

- Built a custom Linux target with Buildroot
- Validated a complete Eclipse-to-target workflow
- Enabled and verified I2C on the embedded target
- Implemented and ran a colour sensor application on Raspberry Pi

Professional value of the project

- Shows embedded Linux system generation, not only application coding
- Demonstrates remote deployment and target-side debugging mindset
- Proves hardware + software integration under a constrained workflow
- Produces a clean, instructor-ready demonstration path

How does it work?

01 Show the sensor

Briefly point to the physical module and mention the four wires: 3V, GND, SDA, SCL.

02 Show I2C detection

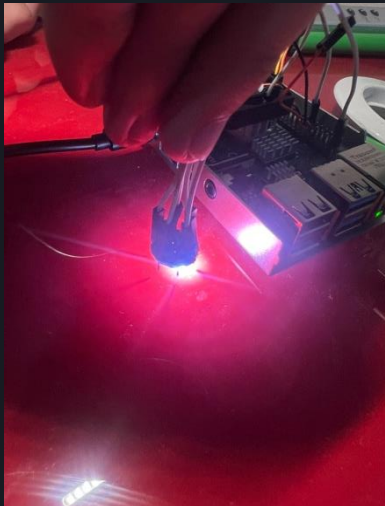
Run `ls /dev/i2c*` and `i2cdetect -y 1` to prove that the bus and address 0x29 are available.

03 Run the final app

Launch `./app 1` on the Raspberry Pi and keep the output visible.

04 Interact with the sensor

Move a coloured object in front of the sensor and comment on the channel changes.



```
Normalized -> R: 50,0% | G: 26,2% | B: 27,9%
Color -> Clear: 12351 | Red: 6166 | Green: 3221 | Blue: 3443
Normalized -> R: 49,9% | G: 26,1% | B: 27,9%

Color -> Clear: 12420 | Red: 6208 | Green: 3237 | Blue: 3445
Normalized -> R: 50,0% | G: 26,1% | B: 27,7%

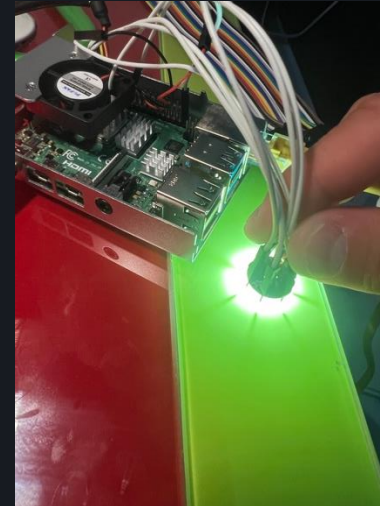
Color -> Clear: 12458 | Red: 6222 | Green: 3249 | Blue: 3475
Normalized -> R: 49,9% | G: 26,1% | B: 27,9%

Color -> Clear: 12435 | Red: 6210 | Green: 3244 | Blue: 3470
Normalized -> R: 49,9% | G: 26,1% | B: 27,9%

Color -> Clear: 12428 | Red: 6207 | Green: 3243 | Blue: 3468
Normalized -> R: 49,9% | G: 26,1% | B: 27,9%

Color -> Clear: 12419 | Red: 6200 | Green: 3241 | Blue: 3467
Normalized -> R: 49,9% | G: 26,1% | B: 27,9%

Color -> Clear: 12418 | Red: 6198 | Green: 3240 | Blue: 3466
Normalized -> R: 49,9% | G: 26,1% | B: 27,9%
```



```
Normalized -> R: 33,7% | G: 40,1% | B: 25,8%
Color -> Clear: 29052 | Red: 9801 | Green: 11660 | Blue: 7513
Normalized -> R: 33,7% | G: 40,1% | B: 25,9%

Color -> Clear: 29054 | Red: 9809 | Green: 11659 | Blue: 7506
Normalized -> R: 33,8% | G: 40,1% | B: 25,8%

Color -> Clear: 29083 | Red: 9810 | Green: 11666 | Blue: 7524
Normalized -> R: 33,7% | G: 40,1% | B: 25,9%

Color -> Clear: 29073 | Red: 9804 | Green: 11663 | Blue: 7532
Normalized -> R: 33,7% | G: 40,1% | B: 25,9%

Color -> Clear: 29039 | Red: 9798 | Green: 11655 | Blue: 7517
Normalized -> R: 33,7% | G: 40,1% | B: 25,9%

Color -> Clear: 28944 | Red: 9773 | Green: 11621 | Blue: 7476
Normalized -> R: 33,8% | G: 40,1% | B: 25,8%

Color -> Clear: 28930 | Red: 9767 | Green: 11614 | Blue: 7473
Normalized -> R: 33,8% | G: 40,1% | B: 25,8%
```

Questions we thought about

Why Buildroot instead of Raspbian?

Because Project II explicitly required a custom embedded Linux workflow, not just application execution on a general-purpose OS.

What was reused from Project I?

The I2C communication pattern: open bus, select slave, read/write registers, periodic acquisition loop and safe termination.

Why did the sensor not work immediately?

At first the Buildroot target did not expose `/dev/i2c-1`, so we had to enable I2C support and rebuild the image.

How do we know the board really sees the sensor?

`i2cdetect -y 1` showed the device at address `0x29` before running the final application.

What proves Chapter 4 is complete?

We compiled from the Buildroot environment, transferred the binary remotely and executed it successfully on the Raspberry Pi target.

THANK YOU FOR YOUR ATTENTION!